Backups

Candex backs up its **production PostgreSQL** database. This handbook covers the strategy, the scripts, how to restore, and how it is automated. For where the database lives, see Database & Model; for the environments, Deployment.

> **Note**
>
> The legacy `candex-backups` repository is the old Firestore-era tool and no longer backs up real data. The current backups are PostgreSQL dumps driven from `candex-api/scripts/backup/`.

Strategy

Only **prod** (`candex_prod`) is backed up — dev is a proving ground. The approach is a daily full dump with grandfather-father-son (GFS) retention, plus a permanent snapshot per release:

- **Daily** full dumps — `KEEP_DAILY = 7`
- **Weekly** — the Sunday dump is promoted — `KEEP_WEEKLY = 5`
- **Monthly** — the 1st-of-month dump is promoted — `KEEP_MONTHLY = 12`
- **Per version** — a labelled snapshot (`--label x.y.z`), **kept forever, never pruned**

Full backups — `daily_backup.sh`

The core script runs a `pg_dump -Fc` (custom format: compressed and selectively restorable) of the whole database, through the running Postgres container:

```
./scripts/backup/daily_backup.sh prod                  # daily full dump
./scripts/backup/daily_backup.sh prod --label 1.1.9    # + a kept version
snapshot
```

Each run writes to `backups/<env>/full/`, validates the dump (it must be non-empty and pass `pg_restore --list`), then promotes it into the `daily`, `weekly`, `monthly`, and `version` tiers as applicable:

```
backups/prod/full/
├── daily/     candex_prod_<timestamp>.dump
├── weekly/    (Sundays)
├── monthly/   (1st of month)
└── version/   candex_prod_v1.1.9.dump   (kept forever)
```

The `backups/` folder is git-ignored — dumps never go into git.

Selective exports — `dump_*.sh`

Alongside the full dumps, per-domain **data-only** exports cover the runtime data that is hard to reconstruct — `dump_catalog_data.sh`, `dump_dex_data.sh`, `dump_user_data.sh` (and `dump_all_runtime_data.sh`). Static reference data is intentionally excluded, because Flyway seeds it (see Database & Model).

Restore & testing

A backup is only as good as a tested restore. `restore.sh` restores a dump into a **throwaway** database so you can verify it, then drop it:

```
./scripts/backup/restore.sh <file.dump> candex_restore_test
# verify, then: dropdb candex_restore_test
```

`pg_restore` must be the same version as the server or newer.

Automation (CI)

Backups run on the self-hosted runner that sits on the production server, via GitHub Actions:

Workflow	When	What
`backup-prod.yml`	Daily at **03:00 UTC** (and manual)	`daily_backup.sh prod`
`deploy-prod.yml`	Before every production deploy	`daily_backup.sh prod --label <version>` — a pre-deploy recovery point

No OS cron is needed — the schedule lives in the workflow.

Storage & off-site

Dumps are kept **on the server** for now; getting them off-host (a second disk or `rsync` to another machine) is a planned improvement. A standalone variant under `ops/backup/` (a `systemd` timer plus `rsync` off-site copy and failure alerting) is available for that server-side setup.

Conventions

- **No secrets in git** — the Postgres password lives in `~/.pgpass` (`chmod 600`); local config (`backup.conf`) and the `backups/` dumps are never committed.
- **Full + selective** — full `-Fc` dumps are the restore path; the per-domain exports are convenience snapshots.
- **Test restores** — a dump that has never been restored is not a backup.

Where to go next

- Where the data lives → Database & Model
- Environments & CI → Deployment